🚀 Streamlit Complete Guide

A structured reference with descriptions, arguments & working examples

📦 1. Installation

Install Streamlit via pip. Requires Python 3.8+

```
pip install streamlit
```

▶️ 2. Run App

Start the Streamlit dev server. Opens at **localhost:8501** by default. You can also run directly from a URL.

```
streamlit run app.py                    # Run local file
streamlit run https://example.com/app.py    # Run from URL
```

📥 3. Imports

Standard imports used throughout most Streamlit apps.

```
import streamlit as st    # Core library
import numpy as np        # Numerical operations
import pandas as pd       # DataFrames
import time               # Used for delays, spinners, progress
import random
```

🖊️ 4. Basic Output

Streamlit provides multiple text display functions, each with different styling.
st.write() is the most versatile — it accepts text, DataFrames, dicts, charts, and more.

st.title(body) — Large top-level heading
st.header(body) — Section heading
st.subheader(body) — Smaller section heading
st.text(body) — Fixed-width plain text
st.write(*args) — Smart display for almost any object

```
st.title("My App Title")
st.header("Section Header")
st.subheader("Sub Section")
st.text("Plain fixed-width text")
st.write("Hello, World!")
st.write("Supports **markdown** too!")
```

▸ Live Output

My App Title

Section Header

Sub Section

Plain fixed-width text

Hello, World!

Supports **markdown** too!

📊 5. Data Display

Display tabular data using **st.dataframe()** (interactive, scrollable) or **st.table()** (static). **st.write(df)** also renders DataFrames interactively.

st.dataframe(data, width, height, use_container_width)
st.table(data) — Static non-scrollable table
st.write(df) — Auto-detects DataFrame and renders interactively

```
df = pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
})

st.write("Here's our first attempt at using data to create a table:")
st.write(df)                              # Interactive table via st.write
st.dataframe(df, use_container_width=True)   # Explicit interactive dataframe
st.table(df)                              # Static table
```

▸ Live Output

Here's our first attempt at using data to create a table:

	first column	second column
0	1	
1	2	
2	3	
3	4	

	first column	second column
0	1	
1	2	
2	3	
3	4	

first column	second column
1	10
2	20
3	30
4	40

🎛️ 6. Widgets

Widgets let users interact with your app. Every widget returns a value that you can use in your Python code. Streamlit reruns the script from top to bottom on every interaction.

6.1 — st.slider()

st.slider(label, min_value, max_value, value, step, key)
• label — text shown above slider
• min_value / max_value — range bounds
• value — default value (or tuple for range)
• step — increment size

```
age = st.slider("Select your age", min_value=0, max_value=100, value=25, step=1)
st.write(f"Your age: {age}")
```

▸ Live Output

Select your age

Your age: 25

6.2 — st.text_input()

st.text_input(label, value, placeholder, type, key)
• type='password' — hides input
• placeholder — ghost text when empty

```
name = st.text_input("Enter your name", placeholder="e.g. Alice")
st.write(f"Hello, {name}!")
```

▸ Live Output

Enter your name

e.g. Alice

Waiting for input...

6.3 — st.number_input()

st.number_input(label, min_value, max_value, value, step, format, key)
- format — e.g. '%0.2f' for 2 decimals

```
num = st.number_input("Enter a number", min_value=0, max_value=1000, value=50, step=5)
st.write(f"You entered: {num}")
```

▸ Live Output

Enter a number

50 − +

You entered: 50

6.4 — st.selectbox()

st.selectbox(label, options, index, format_func, key)
- options — list or tuple of choices
- index — default selected index
- format_func — function to format each option label

```
color = st.selectbox("Pick a color", ["Red", "Green", "Blue"], index=0)
st.write(f"You chose: {color}")
```

▸ Live Output

Pick a color

Red

You chose: Red

6.5 — st.multiselect()

st.multiselect(label, options, default, key)
- default — list of pre-selected values
- Returns a list of selected items

```
skills = st.multiselect("Select your skills", ["Python", "SQL", "ML", "Streamlit"], default=["Python"])
st.write(f"Skills: {skills}")
```

▸ Live Output

Select your skills

Python ×

Skills: ['Python']

6.6 — st.checkbox()

st.checkbox(label, value, key)
- value — default checked state (True/False)
- Returns True if checked, False otherwise

```
agree = st.checkbox("I agree to the terms", value=False)
if agree:
    st.success("Thank you for agreeing!")
```

▸ Live Output

☐ I agree to the terms

6.7 — st.radio()

st.radio(label, options, index, horizontal, key)
- horizontal=True — displays options side by side
- Returns the selected option value

```
plan = st.radio("Choose a plan", ["Free", "Pro", "Enterprise"], horizontal=True)
st.write(f"Selected plan: {plan}")
```

▸ Live Output

Choose a plan

◉ Free ○ Pro ○ Enterprise

Selected plan: Free

6.8 — st.button()

st.button(label, type, use_container_width, key)
- type='primary' — blue filled button; 'secondary' — outline
- Returns True only on the run triggered by the click

```
if st.button("Submit", type="primary"):
    st.write("Button clicked! ✅")
```

▸ Live Output

Submit

📌 7. Sidebar

Add any widget to the sidebar by prefixing with **st.sidebar.** — great for navigation, filters, and settings that persist across the app.

st.sidebar.widget(...) — mirrors every widget available in the main area
with st.sidebar: — context manager style

```
# Method 1: Direct prefix
option = st.sidebar.selectbox("Choose dataset", ["Iris", "Titanic", "MNIST"])

# Method 2: Context manager
with st.sidebar:
    st.header("Settings")
    threshold = st.slider("Threshold", 0.0, 1.0, 0.5)
```

Sidebar selected: `Iris` | Threshold: `0.5`

📐 8. Layout

Organize content using **columns**, **tabs**, **expanders**, and **containers** for clean multi-panel interfaces.

st.columns(spec) — spec is int (equal cols) or list of weights e.g. [2,1]
st.tabs(list) — returns list of tab contexts
st.expander(label, expanded) — collapsible section
st.container() — logical grouping block

```
# Columns
col1, col2, col3 = st.columns([2, 1, 1])
with col1:
    st.metric("Revenue", "$12,500", "+8%")
with col2:
    st.metric("Users", "1,240", "+12%")
with col3:
    st.metric("Churn", "3.2%", "-1%")

# Tabs
tab1, tab2 = st.tabs(["📈 Chart", "📋 Data"])
with tab1:
    st.line_chart(data)
with tab2:
    st.dataframe(data)

# Expander
with st.expander("🔍 See details", expanded=False):
    st.write("Hidden content revealed on click!")
```

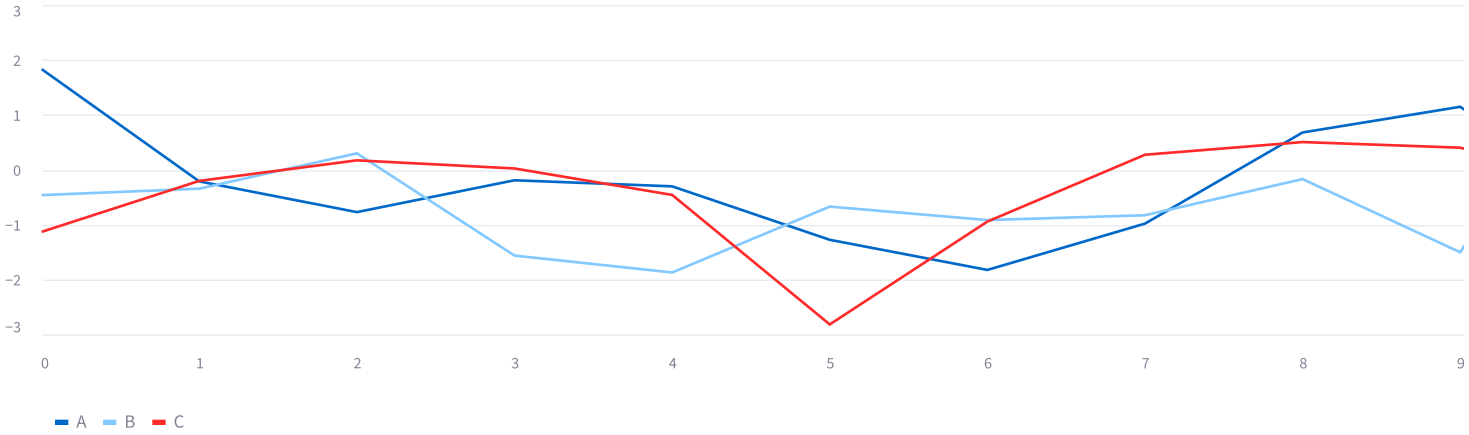
▸ Live Output

Revenue
$12,500
↑ +8%

Users
1,240
↑ +12%

Churn
3.2%
↓ -1%

📈 Chart 📋 Data



🔍 See details

📂 9. File Upload

Allow users to upload files. The returned object behaves like a file-like object — read with **.read()**, load with pandas, etc.

st.file_uploader(label, type, accept_multiple_files, key)
• type — list of allowed extensions e.g. ['csv', 'xlsx']
• accept_multiple_files=True — returns a list of files

```
file = st.file_uploader("Upload a CSV file", type=["csv"])
if file is not None:
    df = pd.read_csv(file)
    st.write(f"Filename: {file.name} | Size: {file.size} bytes")
    st.dataframe(df)
```

▸ Live Output

Upload a CSV file

Upload 200MB per file • CSV

📉 10. Charts

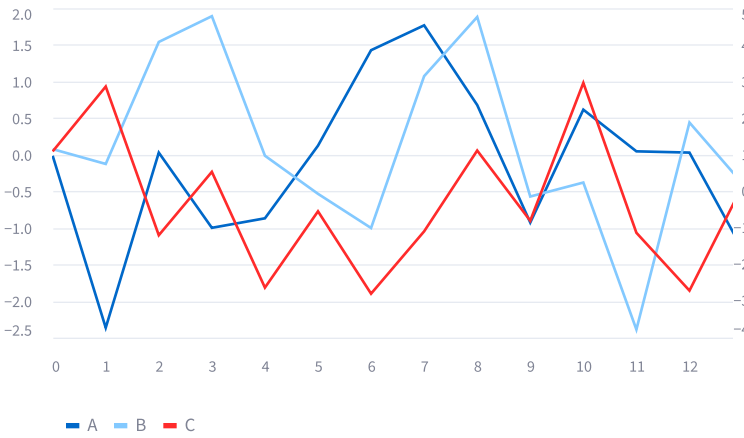
Streamlit includes built-in chart functions. For advanced charts, use **Plotly**, **Altair**, or **Matplotlib** with **st.plotly_chart()**, **st.altair_chart()**, **st.pyplot()**.

st.line_chart(data, x, y, color, use_container_width)
st.bar_chart(data)
st.area_chart(data)
st.scatter_chart(data)

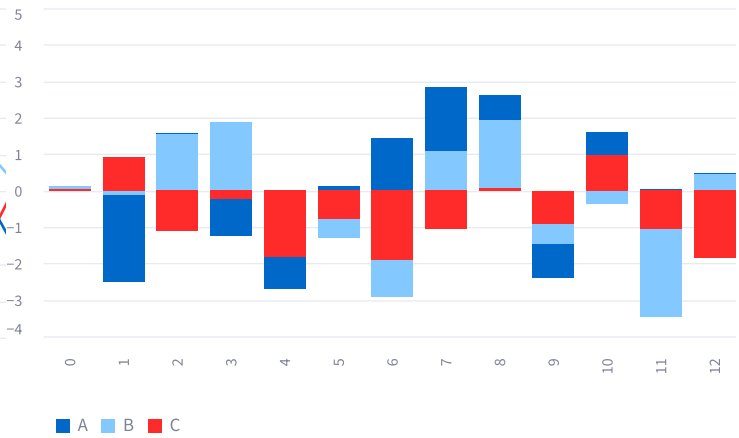
```
data = pd.DataFrame(np.random.randn(20, 3), columns=['A', 'B', 'C'])

st.line_chart(data)
st.bar_chart(data)
st.area_chart(data)
```

▸ Live Output

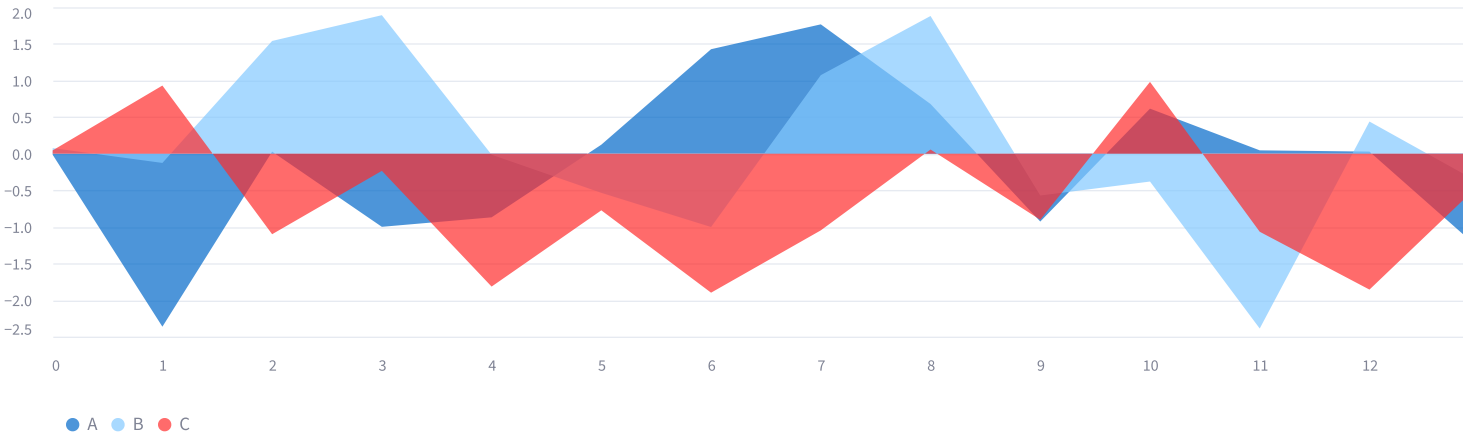
Line Chart



Bar Chart



Area Chart



⏳ 11. Progress Bar

Show task completion progress. Create with **st.progress()** and update dynamically in a loop using **.progress(value)**. Value ranges from 0 to 100 (or 0.0 to 1.0).

st.progress(value, text)
- value — int (0–100) or float (0.0–1.0)
- text — optional label shown beside the bar

```
bar = st.progress(0, text="Processing...")

for i in range(100):
    bar.progress(i + 1, text=f"Step {i+1}/100")
```

```
    time.sleep(0.01)

st.success("Done!")
```

▸ Live Output

▶ Run Progress Demo

🔔 12. Status Messages

Display color-coded alert boxes to communicate state, errors, warnings, or general info to users.

st.success(body, icon) — green box
st.error(body, icon) — red box
st.warning(body, icon) — yellow box
st.info(body, icon) — blue box
st.exception(e) — shows traceback

```
st.success("✅ Data loaded successfully!")
st.error("❌ Failed to connect to database.")
st.warning("⚠️ API rate limit approaching.")
st.info("ℹ️ Results are cached for 10 minutes.")
```

▸ Live Output

✅ Data loaded successfully!

❌ Failed to connect to database.

⚠️ API rate limit approaching.

ℹ️ Results are cached for 10 minutes.

🌀 13. Spinner

Show a loading animation while a long operation runs. Use as a context manager — the spinner disappears when the **with** block exits.

with st.spinner(text)
• text — message displayed beside the spinner animation

```
with st.spinner("Fetching data from API..."):
    time.sleep(2)       # simulate delay
    data = fetch_data()
```

```
st.success("Data loaded!")
```

▸ Live Output

▶ Run Spinner Demo

💾 14. Session State

Streamlit reruns the entire script on every interaction, resetting all variables. **st.session_state** persists values across reruns — essential for counters, multi-step forms, login state, etc. Access like a dict: `st.session_state['key']` or like an attribute: `st.session_state.key`

st.session_state[key] — dict-style access
st.session_state.key — attribute-style access
• Initialize with: `if 'key' not in st.session_state: st.session_state.key = default`

```
# Initialize counter if not yet set
if "count" not in st.session_state:
    st.session_state.count = 0

col1, col2, col3 = st.columns(3)
if col1.button("➕ Increment"):
    st.session_state.count += 1
if col2.button("➖ Decrement"):
    st.session_state.count -= 1
if col3.button("🔄 Reset"):
    st.session_state.count = 0

st.metric("Counter", st.session_state.count)
```

▸ Live Output

➕ Increment ➖ Decrement 🔄 Reset

Counter Value

0

⚡ 15. Caching

Avoid re-running expensive computations on every rerun.
• **@st.cache_data** — for data (DataFrames, API responses, computed values). Each call returns a copy.
• **@st.cache_resource** — for shared resources (ML models, DB connections). Returns the same object.

@st.cache_data(ttl, max_entries, show_spinner)
• ttl — time-to-live in seconds (e.g. ttl=3600)
• max_entries — max cached results to keep
@st.cache_resource(ttl, show_spinner)

```python
@st.cache_data(ttl=600)            # Cache for 10 minutes
def load_data(url: str):
    return pd.read_csv(url)

@st.cache_resource                 # Load model once, share across sessions
def load_model():
    return MyMLModel.load("model.pkl")

df = load_data("https://data.example.com/data.csv")
model = load_model()
```

▸ Live Output (cached function)

	x	y
0	0	
1	1	
2	2	
3	3	
4	4	

📝 16. Forms

Wrap widgets in a form so the app only reruns when the user explicitly submits — preventing premature reruns on each widget interaction.

with st.form(key, clear_on_submit)
• key — unique form identifier (required)
• clear_on_submit — reset widgets after submit
st.form_submit_button(label, type) — must be inside the form

```python
with st.form("signup_form", clear_on_submit=False):
    st.subheader("Sign Up")
    username = st.text_input("Username")
    email    = st.text_input("Email")
    role     = st.selectbox("Role", ["Developer", "Analyst", "Manager"])
    submit   = st.form_submit_button("Register", type="primary")

if submit:
    st.success(f"Registered: {username} | {email} | {role}")
```

▸ Live Output

Sign Up

Username

Email

Role

Developer

Register

💬 17. Chat Interface

Build conversational UIs with **st.chat_input()** and **st.chat_message()**. Combine with session state to maintain conversation history.

st.chat_input(placeholder, key) — returns typed message or None
st.chat_message(name) — name is 'user' or 'assistant' (or any string)
• Use inside a **with** block to write the message content

```
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display history
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# Accept new input
if prompt := st.chat_input("Type a message..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.write(prompt)

    reply = f"Echo: {prompt}"
    st.session_state.messages.append({"role": "assistant", "content": reply})
    with st.chat_message("assistant"):
        st.write(reply)
```

▸ Live Output

🎨 18. HTML & Styling

Use **st.markdown()** with **unsafe_allow_html=True** to inject custom HTML and CSS for advanced styling beyond Streamlit's defaults.

st.markdown(body, unsafe_allow_html)
• body — markdown string or HTML string
• unsafe_allow_html=True — required to render raw HTML tags
st.html(body) — (Streamlit 1.31+) dedicated HTML renderer

```
st.markdown("<h3 style='color:#0f62fe;'>Blue Heading</h3>", unsafe_allow_html=True)

st.markdown("""
<div style='background:#1e3a5f; color:white; padding:12px; border-radius:8px;'>
    Custom styled card content
```

```
</div>
""", unsafe_allow_html=True)

# Inject global CSS
st.markdown("""
<style>
    .stButton > button { background: #0f62fe; color: white; border-radius: 8px; }
</style>
""", unsafe_allow_html=True)
```

▸ Live Output

Blue Heading

✨ Custom styled card — use HTML for anything Streamlit can't do natively.

🌊 19. Streaming — st.write_stream()

st.write_stream() consumes a Python generator (or any iterable) and prints each yielded chunk to the screen as it arrives — producing a live typing / streaming effect.

This is the standard way to stream LLM responses (OpenAI, Anthropic, etc.) in Streamlit. The function returns the full assembled string once streaming completes.

st.write_stream(stream)
- stream — a generator, iterator, or any iterable that yields string chunks
- Also accepts: OpenAI stream objects, Anthropic stream objects directly
- Returns: the full concatenated string after streaming finishes

```
import time

# 1. Basic generator-based stream
def stream_text(text, delay=0.03):
    for word in text.split(" "):
        yield word + " "
        time.sleep(delay)

if st.button("▶ Stream Text"):
    response = st.write_stream(stream_text("Hello! This text streams word by word."))
    # response now holds the full string


# 2. Character-by-character (slower, typewriter feel)
def typewriter(text, delay=0.02):
    for char in text:
        yield char
        time.sleep(delay)

if st.button("▶ Typewriter Effect"):
    st.write_stream(typewriter("Typing... one... character... at... a... time."))


# 3. Simulated LLM chunk streaming
def fake_llm_stream(prompt):
    reply = f"You asked: '{prompt}'. Here is a streamed answer with variable chunk sizes."
    words = reply.split()
    for i, word in enumerate(words):
```

```
        chunk_size = random.randint(1, 3)        # simulate variable chunk sizes
        yield " ".join(words[i:i+chunk_size]) + " "
        time.sleep(random.uniform(0.03, 0.08))   # simulate network jitter
        if i + chunk_size >= len(words):
            break

if st.button("▶ Fake LLM Stream"):
    st.write_stream(fake_llm_stream("What is Streamlit?"))


# 4. Real OpenAI streaming (requires: pip install openai)
import openai
client = openai.OpenAI(api_key="YOUR_API_KEY")

def openai_stream(prompt):
    stream = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )
    for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            yield delta

if st.button("▶ OpenAI Stream"):
    st.write_stream(openai_stream("Explain Streamlit in 2 sentences."))
```

▸ Live Output

🌊 Word Stream ⌨️ Typewriter 🤖 Fake LLM

💬 20. Streaming Inside Chat Messages

> The most common real-world pattern: stream the assistant reply *inside* a **st.chat_message()** block. Use **st.session_state** to store the full response after streaming so it can be re-rendered on reruns without re-streaming.

```
import time, random

def fake_llm_stream(prompt):
    reply = f"Sure! Here's my answer to '{prompt}': Streamlit streams responses word by word."
    for word in reply.split():
        yield word + " "
        time.sleep(random.uniform(0.04, 0.1))

if "stream_chat" not in st.session_state:
    st.session_state.stream_chat = []

# Render existing messages
for msg in st.session_state.stream_chat:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# New input
if prompt := st.chat_input("Ask something..."):
    # Show & store user message
    st.session_state.stream_chat.append({"role": "user", "content": prompt})
```

```
    with st.chat_message("user"):
        st.write(prompt)

    # Stream assistant reply
    with st.chat_message("assistant"):
        full_reply = st.write_stream(fake_llm_stream(prompt))   # streams live

    # Store the completed reply for future reruns
    st.session_state.stream_chat.append({"role": "assistant", "content": full_reply})
```

▸ Live Output

🗑️ Clear Chat

📦 21. Manual Typing with st.empty()

st.empty() creates a single placeholder slot that can be *overwritten* repeatedly. This gives you full control over the typing effect — useful when you want custom styling, a blinking cursor, markdown rendering mid-stream, or any non-standard display.

placeholder = st.empty()
• placeholder.write(x) — overwrite with text / markdown
• placeholder.markdown(x) — overwrite with styled markdown
• placeholder.empty() — clear the slot completely
• Works with: write, markdown, code, dataframe, image, and more

```
import time

placeholder = st.empty()
full_text = "This text appears character by character using st.empty()!"

# Method A — character by character with blinking cursor
typed = ""
for char in full_text:
    typed += char
    placeholder.markdown(typed + "▌")   # ▌ acts as blinking cursor
    time.sleep(0.03)
placeholder.markdown(typed)             # remove cursor at end


# Method B — word by word with custom styled container
words = "Streaming words with bold **markdown** and `code` rendered live!".split()
built = ""
for word in words:
    built += word + " "
    placeholder.markdown(f"> {built}▌")
    time.sleep(0.07)
placeholder.markdown(f"> {built}")


# Method C — overwrite with completely different content when done
placeholder.success("✅ Streaming complete!")
```

▸ Live Output

⌨️ Char + Cursor 📝 Word + Markdown ✅ Then Replace

🔄 22. Live Step Updates with st.status()

st.status() shows an expandable container that displays live step-by-step progress while a long task runs. It starts in a *running* state and transitions to *complete* or *error* when you call **.update()**. Perfect for multi-step pipelines, data loading, or agentic workflows.

with st.status(label, expanded, state) as status:
• label — text shown in the header bar
• expanded=True — shows steps as they run
• state — 'running' | 'complete' | 'error'
• status.update(label, state, expanded) — call at the end to finalise

```
import time

with st.status("⚙️ Processing pipeline...", expanded=True) as status:
    st.write("🔍 Step 1: Validating inputs...")
    time.sleep(1)

    st.write("📥 Step 2: Loading data...")
    time.sleep(1.5)

    st.write("🧠 Step 3: Running model inference...")
    time.sleep(2)

    st.write("💾 Step 4: Saving results...")
    time.sleep(1)

    status.update(label="✅ Pipeline complete!", state="complete", expanded=False)

st.success("All steps finished successfully.")
```

▸ Live Output

▶ Run Pipeline (success)

▶ Run Pipeline (error)

⚡ 23. st.fragment — Partial Re-runs

By default, Streamlit reruns the *entire* script on every interaction. **@st.fragment** lets you isolate a function so only that section reruns — leaving the rest of the page untouched. This is essential for smooth streaming and live-updating components inside larger apps.

@st.fragment(run_every=None)
• Decorate a function — only that function reruns on interaction inside it
• run_every — auto-rerun interval e.g. `'5s'`, `'1m'`, or seconds as float
• Requires Streamlit ≥ 1.33

```
import streamlit as st
import time, random

# This fragment reruns independently — the rest of the page stays frozen
```

```python
@st.fragment
def live_ticker():
    cols = st.columns(3)
    cols[0].metric("BTC", f"${random.randint(60000,70000):,}", f"{random.uniform(-2,2):.2f}%")
    cols[1].metric("ETH", f"${random.randint(3000,4000):,}",  f"{random.uniform(-2,2):.2f}%")
    cols[2].metric("SOL", f"${random.randint(130,180):,}",    f"{random.uniform(-2,2):.2f}%")
    if st.button("🔄 Refresh Prices"):
        pass   # clicking this only reruns live_ticker(), not the whole page

live_ticker()


# Auto-refreshing fragment — updates every 3 seconds automatically
@st.fragment(run_every="3s")
def auto_clock():
    st.write(f"🕐 Current time: {time.strftime('%H:%M:%S')}")

auto_clock()
```

▸ Live Output

👇 Clicking Refresh only reruns this fragment — not the whole page

BTC
$60,338
↑ 0.64%

ETH
$3,877
↑ 1.96%

SOL
$178
↓ -0.98%

🔄 Refresh Prices

🕐 Auto-refreshes every 2s: **19:26:47**

📋 Quick Reference — Streaming Cheatsheet

Choose the right tool:

Use Case	Tool
Stream LLM / generator output	`st.write_stream(gen)`
Custom typewriter with cursor/styling	`st.empty()` + overwrite
Multi-step pipeline progress	`st.status()`
Isolate a section from full reruns	`@st.fragment`
Auto-refresh a section periodically	`@st.fragment(run_every="Xs")`
Stream inside chat bubbles	`st.chat_message` + `st.write_stream`

```python
# Minimal streaming pattern — copy & paste ready
import streamlit as st
import time

def my_stream(text):
    for word in text.split():
        yield word + " "
        time.sleep(0.05)
```

```python
if st.button("Stream"):
    result = st.write_stream(my_stream("Hello streaming world!"))
    st.write("Full text:", result)   # result holds the assembled string
```

⚡ Streamlit Streaming Extension | Append to your existing guide

🚀 Streamlit Complete Guide | Happy Building!

Type a message...

Ask something and watch it stream...